It is interesting that only 4% of these passwords were dictionary words and the great majority is actually alphanumeric. However, password cracker dictionaries contain a large number of today's passwords, and they try out all kinds of (alphanumerical) combinations. If an attacker knows your user name and you use a weak password, your account will be easily cracked.

A good password is a long alphanumeric combination of mixed cases. As this is quite hard to remember, it is advisable to enter only the *first letters of a sentence that you can easily remember*. For example "The quick brown fox jumps over the lazy dog" will be "Tqbfjotld". Note that this is just an example, you should not use well known phrases like these, as they might appear in cracker dictionaries, too.

6.6 Regular Expressions

A common pitfall in Ruby's regular expressions is to match the string's beginning and end by ^ and $, instead of \A and \z.

Ruby uses a slightly different approach than many other languages to match the end and the beginning of a string. That is why even many Ruby and Rails books get this wrong. So how is this a security threat? Say you wanted to loosely validate a URL field and you used a simple regular expression like this:

```
/^https?:\/\/[^\n]+$/i
```

This may work fine in some languages. However, *in Ruby ^ and $ match the **line** beginning and line end*. And thus a URL like this passes the filter without problems:

```
javascript:exploit_code();/*
http://hi.com
*/
```

This URL passes the filter because the regular expression matches - the second line, the rest does not matter. Now imagine we had a view that showed the URL like this:

```
link_to "Homepage", @user.homepage
```

The link looks innocent to visitors, but when it's clicked, it will execute the JavaScript function "exploit_code" or any other JavaScript the attacker provides.

To fix the regular expression, \A and \z should be used instead of ^ and $, like so:

```
/\Ahttps?:\/\/[^\n]+\z/i
```

Since this is a frequent mistake, the format validator (validates_format_of) now raises an exception if the provided regular expression starts with ^ or ends with $. If you do need to use ^ and $ instead of \A and \z (which is rare), you can set the :multiline option to true, like so:

```
# content should include a line "Meanwhile" anywhere in the string
validates :content, format: { with: /^Meanwhile$/, multiline: true }
```

Note that this only protects you against the most common mistake when using the format validator - you always need to keep in mind that ^ and $ match the **line** beginning and line end in Ruby, and not the beginning and end of a string.

6.7 Privilege Escalation

Changing a single parameter may give the user unauthorized access. Remember that every parameter may be changed, no matter how much you hide or obfuscate it.